
TEST PLAN: PETSTORE WEB APPLICATION

1. Introduction

This Test Plan document defines the overall testing approach for the Petstore web application. The purpose of this document is to establish a structured and systematic plan that governs all testing activities related to the application. It describes the scope, objectives, strategy, environment, responsibilities, and deliverables associated with the testing process.

The Test Plan acts as a single point of reference for stakeholders, testers, and reviewers to understand how testing will be performed, what will be tested, and under which conditions testing activities will be considered complete. This document follows standard Software Testing Life Cycle (STLC) practices and aligns with industry-recognized quality assurance methodologies.

The goal of this test plan is to ensure that the Petstore application meets its functional expectations, provides a stable user experience, and behaves consistently across its core workflows.

2. Application Overview

The Petstore web application is a demo-based e-commerce platform designed to simulate an online pet store. The application allows end users to browse different categories of pets, explore available products, add selected items to a shopping cart, update cart contents, and complete the checkout process.

The application supports the following pet categories:

- Bird
- Reptile
- Cat
- Dog
- Fish

Each category contains multiple products that users can view and interact with. The application is web-based, publicly accessible, and intended primarily for demonstration, testing, and learning purposes. Although it is a demo environment, the application represents a real-world e-commerce workflow and provides an ideal platform for functional testing.

3. Test Objectives

The primary objectives of testing the Petstore web application are as follows:

- To validate that all core functional features operate as intended
- To ensure that users can successfully navigate through the application without encountering errors
- To verify that product browsing, cart management, and checkout processes work correctly
- To identify functional defects and inconsistencies in application behavior
- To ensure the application delivers a reliable and user-friendly experience
- To confirm that the application is functionally stable within the defined scope

These objectives guide the overall testing effort and help ensure that testing activities are aligned with business and quality expectations.

4. Test Scope

4.1 In Scope

The following functional areas are included within the scope of testing:

- Browsing available pet categories
- Viewing product listings within categories
- Viewing detailed product information
- Adding products to the shopping cart
- Updating shopping cart contents (quantity changes and item removal)
- Proceeding through the checkout workflow
- Basic navigation and page transition validation

Testing will focus on validating user-visible functionality and end-to-end workflows from a functional perspective.

4.2 Out of Scope

The following areas and testing types are excluded from the scope of this test plan:

- Performance, load, and stress testing
 - Security and vulnerability testing
 - Automation testing
 - Backend, database, or API validation
 - Mobile or cross-browser testing beyond the selected browser
 - Integration testing with third-party systems
-

5. Test Strategy

The testing strategy is based on manual functional testing and focuses on validating application behavior against expected user workflows.

The strategy includes:

- Designing detailed test cases based on functional requirements and user actions
- Executing test cases on the live application
- Comparing actual results with expected results
- Logging execution results and observations in TestRail

Test execution results are classified as:

- **Pass:** The actual result matches the expected result
- **Fail:** The actual result does not match the expected result
- **Blocked:** Test execution is not possible due to environment or dependency issues
- **Skipped:** Test is intentionally not executed under specific conditions

This approach ensures consistency, traceability, and transparency throughout the testing process.

6. Test Environment

Testing will be conducted in the following environment:

Component	Description
Application URL	https://petstore.octoperf.com
Browser	Google Chrome
Operating System	Windows
Internet Connectivity	Stable broadband connection
Test Management Tool	TestRail
Testing Type	Manual Functional Testing

The selected environment is considered sufficient to validate the application's core functionality.

7. Test Deliverables

The following deliverables will be produced during the testing lifecycle:

- Approved Test Plan document
- Functional test cases managed in TestRail
- Test execution reports
- Test result summaries
- Test closure and summary report

These deliverables provide traceable evidence of testing activities and outcomes.

8. Entry Criteria

Testing activities will begin only when the following conditions are satisfied:

- The application is accessible and stable
 - Test cases have been designed and reviewed
 - The test environment is available and configured
 - Required testing tools are accessible
-

9. Exit Criteria

Testing will be considered complete when:

- All planned test cases have been executed
 - Test execution results are documented
 - Major functional issues have been identified and recorded
 - A test summary report has been prepared
 - No critical unresolved issues remain within the defined scope
-

10. Roles & Responsibilities

Role	Responsibility
QA Engineer	Test planning, test case design, execution, reporting
Stakeholder	Review and approval of test results

11. Assumptions

The following assumptions apply to this test plan:

- The application will remain available throughout the testing period
 - Test data will be available when required
 - No major functional changes will be introduced during execution
 - Internet connectivity will remain stable
 - The demo environment will behave consistently
-

12. Risks & Mitigation

Risk	Mitigation
Application downtime	Pause testing and resume when available
Limited control over demo environment	Clearly document observed issues
Test data reset	Re-execute affected test cases
Environment instability	Mark impacted tests as blocked